

Top 100 Python Interview Questions

Data Structures: Lists · Dicts · Sets · Tuples

Section 1: Lists (Q1–Q30)

Q1. What is a Python list and how is it different from a tuple?

Answer: A list is a mutable, ordered collection of elements enclosed in square brackets. Unlike tuples, lists support in-place modification (append, remove, etc.).

Example:

```
my_list = [1, 2, 3]      # list - mutable
my_tuple = (1, 2, 3)     # tuple - immutable
my_list[0] = 99          # OK
# my_tuple[0] = 99       # TypeError
```

Q2. How do you access elements of a list using negative indexing?

Answer: Negative indices count from the end of the list: -1 is the last element, -2 is second-to-last, and so on.

Example:

```
nums = [10, 20, 30, 40, 50]
print(nums[-1]) # 50
print(nums[-2]) # 40
print(nums[-5]) # 10
```

Q3. Explain list slicing with examples.

Answer: Slicing uses list[start:stop:step]. start is inclusive, stop is exclusive. Omitting start/stop defaults to 0/end.

Example:

```
nums = [0, 1, 2, 3, 4, 5]
print(nums[1:4]) # [1, 2, 3]
print(nums[:3])  # [0, 1, 2]
print(nums[::2]) # [0, 2, 4]
print(nums[::-1]) # [5, 4, 3, 2, 1, 0] (reversed)
```

Q4. How do you add elements to a list?

Answer: Use append() to add to the end, insert() to add at a specific index, or extend() / '+=' to add multiple elements.

Example:

```
lst = [1, 2, 3]
lst.append(4)      # [1, 2, 3, 4]
lst.insert(1, 10)  # [1, 10, 2, 3, 4]
lst.extend([5, 6]) # [1, 10, 2, 3, 4, 5, 6]
```

Q5. How do you remove elements from a list?

Answer: `remove()` deletes the first occurrence of a value; `pop()` removes by index (returns the removed element); `del` removes by index/slice.

Example:

```
lst = [1, 2, 3, 2, 4]
lst.remove(2)      # [1, 3, 2, 4] - removes first 2
val = lst.pop(1)   # val=3, lst=[1, 2, 4]
del lst[0]         # [2, 4]
```

Q6. What is the difference between `copy()` and `deepcopy()` for lists?

Answer: `copy()` (or `list[:]`) creates a shallow copy – nested objects are still shared. `deepcopy()` from the `copy` module recursively copies all nested objects.

Example:

```
import copy
orig = [[1, 2], [3, 4]]
shallow = orig.copy()
deep = copy.deepcopy(orig)
orig[0][0] = 99
print(shallow[0][0]) # 99 (shared reference)
print(deep[0][0])   # 1 (independent copy)
```

Q7. How do you sort a list in Python?

Answer: `list.sort()` sorts in-place; `sorted()` returns a new sorted list. Both accept `key=` and `reverse=` arguments.

Example:

```
nums = [3, 1, 4, 1, 5]
nums.sort() # [1, 1, 3, 4, 5] in-place
words = ['banana', 'apple', 'cherry']
print(sorted(words, key=len)) # ['apple', 'banana', 'cherry']
print(sorted(nums, reverse=True)) # [5, 4, 3, 1, 1]
```

Q8. How do you find the index of an element in a list?

Answer: `index(value)` returns the first occurrence. Pass `start` and `stop` to restrict the search range. Use `enumerate()` when you need all indices.

Example:

```
lst = [10, 20, 30, 20, 40]
print(lst.index(20)) # 1
print(lst.index(20, 2)) # 3 (search from pos 2)
all_idx = [i for i, v in enumerate(lst) if v == 20]
print(all_idx) # [1, 3]
```

Q9. What are list comprehensions and when should you use them?

Answer: List comprehensions provide a concise way to create lists. Use them for simple transformations/filtering; prefer a regular loop for complex logic.

Example:

```
squares = [x**2 for x in range(10)]
evens = [x for x in range(20) if x % 2 == 0]
pairs = [(x, y) for x in [1,2] for y in [3,4]]
```

```
# pairs → [(1,3),(1,4),(2,3),(2,4)]
```

Q10. How do you flatten a nested list?

Answer: Use a nested list comprehension for one level of nesting, or `itertools.chain.from_iterable()` for a flat iterable approach.

Example:

```
nested = [[1, 2], [3, 4], [5]]
flat = [x for sub in nested for x in sub]
print(flat) # [1, 2, 3, 4, 5]

import itertools
flat2 = list(itertools.chain.from_iterable(nested))
```

Q11. What is the time complexity of common list operations?

Answer: Python lists are backed by dynamic arrays. Key complexities: access $O(1)$, append $O(1)$ amortised, insert/delete at arbitrary index $O(n)$, search $O(n)$.

Example:

```
lst = list(range(1_000_000))
#  $O(1)$  - index access
x = lst[500_000]
#  $O(1)$  amortised - append
lst.append(99)
#  $O(n)$  - insert at front
lst.insert(0, -1)
```

Q12. How does Python list multiplication work?

Answer: The `*` operator repeats the list. For mutable nested objects this is a shallow repetition – all copies share the same inner references.

Example:

```
a = [0] * 5
print(a) # [0, 0, 0, 0, 0]

b = [[]] * 3 # GOTCHA: three refs to SAME list
b[0].append(1)
print(b) # [[1], [1], [1]] - all changed!

safe = [[] for _ in range(3)] # independent lists
```

Q13. How do you remove duplicates from a list while preserving order?

Answer: Use `dict.fromkeys()` (Python 3.7+ preserves insertion order) or a seen set approach.

Example:

```
lst = [3, 1, 4, 1, 5, 9, 2, 6, 5]
unique = list(dict.fromkeys(lst))
print(unique) # [3, 1, 4, 5, 9, 2, 6]

# Equivalent with set (same order guarantee in 3.7+)
```

```
seen, result = set(), []
[result.append(x) for x in lst if not (x in seen or seen.add(x))]
```

Q14. What is the difference between '==' and 'is' for lists?

Answer: '==' checks value equality; 'is' checks identity (same object in memory).

Example:

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a
print(a == b)    # True - same values
print(a is b)    # False - different objects
print(a is c)    # True - same object
```

Q15. How do you count occurrences of an element in a list?

Answer: list.count(value) returns the number of times value appears. For counting all elements, use collections.Counter.

Example:

```
lst = [1, 2, 3, 2, 1, 2]
print(lst.count(2))    # 3

from collections import Counter
c = Counter(lst)
print(c)    # Counter({2: 3, 1: 2, 3: 1})
```

Q16. How do you zip and unzip lists?

Answer: zip() pairs elements from multiple iterables into tuples. Unzip with zip(*zipped).

Example:

```
names = ['Alice', 'Bob', 'Carol']
scores = [95, 87, 92]
zipped = list(zip(names, scores))
# [('Alice', 95), ('Bob', 87), ('Carol', 92)]

n2, s2 = zip(*zipped)    # unzip
print(list(n2))    # ['Alice', 'Bob', 'Carol']
```

Q17. What does list.reverse() do and how does it differ from reversed()?

Answer: list.reverse() reverses in-place and returns None. reversed() returns a lazy iterator without modifying the original.

Example:

```
lst = [1, 2, 3, 4]
lst.reverse()    # in-place, returns None
print(lst)    # [4, 3, 2, 1]

lst2 = [1, 2, 3, 4]
rev = list(reversed(lst2))    # new list
print(lst2)    # [1, 2, 3, 4] unchanged
```

Q18. How do you check if a list is empty?

Answer: Use 'if not lst:' – an empty list is falsy. Alternatively compare with `len(lst) == 0`.

Example:

```
lst = []
if not lst:
    print('List is empty')    # prints this

# Also valid:
if len(lst) == 0:
    print('Empty')
```

Q19. Explain the difference between `append()` and `extend()`.

Answer: `append()` adds its argument as a single element (even if it's a list). `extend()` iterates over its argument and adds each element individually.

Example:

```
a = [1, 2, 3]
a.append([4, 5])    # [1, 2, 3, [4, 5]] - nested list

b = [1, 2, 3]
b.extend([4, 5])    # [1, 2, 3, 4, 5] - flat
```

Q20. How do you use the `map()` function with lists?

Answer: `map(func, iterable)` applies `func` to every element lazily. Wrap in `list()` to materialise.

Example:

```
nums = [1, 2, 3, 4]
squared = list(map(lambda x: x**2, nums))
print(squared)    # [1, 4, 9, 16]

# Equivalent list comprehension
squared2 = [x**2 for x in nums]
```

Q21. How does `filter()` work with lists?

Answer: `filter(func, iterable)` keeps elements for which `func` returns `True`. Pass `None` as `func` to filter out falsy values.

Example:

```
nums = [0, 1, 2, 3, 4, 5]
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens)    # [0, 2, 4]

mixed = [0, '', None, 1, 'hi', False]
truthy = list(filter(None, mixed))
print(truthy)    # [1, 'hi']
```

Q22. How do you use reduce() on a list?

Answer: `functools.reduce(func, iterable[, initializer])` cumulatively applies a two-argument function to reduce a list to a single value.

Example:

```
from functools import reduce
nums = [1, 2, 3, 4, 5]
total = reduce(lambda a, b: a + b, nums)      # 15
product = reduce(lambda a, b: a * b, nums, 1) # 120
```

Q23. What is the difference between list() and [] for creating lists?

Answer: `[]` is a literal and slightly faster; `list()` is a constructor that can convert any iterable to a list.

Example:

```
a = []          # empty list (literal)
b = list()      # empty list (constructor)
c = list('hello') # ['h','e','l','l','o']
d = list(range(5)) # [0,1,2,3,4]
```

Q24. How do you merge two lists without duplicates?

Answer: Convert to sets and take the union, then convert back to a list.

Example:

```
a = [1, 2, 3, 4]
b = [3, 4, 5, 6]
merged = list(set(a) | set(b))
# order not guaranteed; to preserve a's order:
merged_ordered = a + [x for x in b if x not in set(a)]
```

Q25. How do you rotate a list by k positions?

Answer: Use slicing: rotate left by k with `lst[k:] + lst[:k]`; or use `collections.deque.rotate()`.

Example:

```
from collections import deque
lst = [1, 2, 3, 4, 5]
k = 2
# Rotate left (deque rotates right by default)
d = deque(lst)
d.rotate(-k)      # deque([3, 4, 5, 1, 2])

# Slicing approach
rotated = lst[k:] + lst[:k] # [3, 4, 5, 1, 2]
```

Q26. What is the purpose of the 'in' operator with lists vs sets?

Answer: `'in'` on a list is $O(n)$; on a set it is $O(1)$ average due to hashing. Use sets for repeated membership tests.

Example:

```
import time
data_list = list(range(1_000_000))
data_set = set(data_list)
```

```
# Fast O(1)
999_999 in data_set    # near-instant
# Slow O(n)
999_999 in data_list   # scans entire list
```

Q27. How do you transpose a matrix (list of lists)?

Answer: Use `zip(*matrix)` which unpacks rows as separate arguments to `zip`, which then groups columns.

Example:

```
matrix = [[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9]]
transposed = [list(row) for row in zip(*matrix)]
# [[1,4,7],[2,5,8],[3,6,9]]
```

Q28. How do you find the second-largest element in a list?

Answer: Sort and index `-2`, or use `heapq.nlargest(2)` which is $O(n)$ rather than $O(n \log n)$.

Example:

```
import heapq
nums = [3, 1, 4, 1, 5, 9, 2, 6]
second = heapq.nlargest(2, nums)[1]
print(second)    # 6

# Or with sorted unique values:
second2 = sorted(set(nums))[-2]    # 6
```

Q29. What happens when you use `+=` vs `.extend()` on a list?

Answer: Both mutate in-place and are equivalent for lists. However, `+=` also works on tuples (creating a new object), while `.extend()` is list-only.

Example:

```
lst = [1, 2, 3]
lst += [4, 5]    # same object, id unchanged
print(lst)       # [1, 2, 3, 4, 5]

lst2 = [1, 2, 3]
lst2.extend([4, 5])    # identical result
```

Q30. How do you create a list of a specific size initialized to a default value?

Answer: Use the multiplication operator for immutable defaults. For mutable defaults (like lists), use a comprehension.

Example:

```
zeros = [0] * 5    # [0, 0, 0, 0, 0]
nones = [None] * 3  # [None, None, None]
# Safe mutable defaults:
matrix = [[0]*3 for _ in range(3)]    # 3x3 grid
```

Q31. How do you check if all or any elements satisfy a condition?

Answer: Use `all()` and `any()` with generator expressions for efficient short-circuit evaluation.

Example:

```
nums = [2, 4, 6, 8]
print(all(x % 2 == 0 for x in nums)) # True
print(any(x > 7 for x in nums))     # True
print(all(x > 5 for x in nums))     # False
```

Section 2: Dictionaries (Q31–Q55)

Q32. What is a Python dictionary and what are its key properties?

Answer: A dict is a mutable, ordered (Python 3.7+) mapping of unique keys to values. Keys must be hashable. Average $O(1)$ get/set.

Example:

```
d = {'name': 'Alice', 'age': 30, 'city': 'NY'}
print(d['name']) # 'Alice'
d['score'] = 95  # add/update
print(len(d))   # 4
```

Q33. What is the difference between `dict.get()` and `dict[]`?

Answer: `[]` raises `KeyError` if key is missing; `get(key, default)` returns the default (`None` if not specified) without raising.

Example:

```
d = {'a': 1}
print(d['a']) # 1
# print(d['b']) # KeyError
print(d.get('b')) # None
print(d.get('b', 0)) # 0
```

Q34. How does `dict.setdefault()` work?

Answer: `setdefault(key, default)` returns the value if key exists; otherwise inserts key with default and returns default.

Example:

```
d = {}
d.setdefault('colors', []).append('red')
d.setdefault('colors', []).append('blue')
print(d) # {'colors': ['red', 'blue']}
```

Q35. How do you merge two dictionaries?

Answer: In Python 3.9+ use the `|` operator. In 3.5+ use `**a, **b` unpacking. Or use `dict.update()`.

Example:

```
a = {'x': 1, 'y': 2}
```



```

b = {'y': 99, 'z': 3}

merged = a | b          # 3.9+  {'x':1,'y':99,'z':3}
merged = {**a, **b}     # 3.5+  same result
a.update(b)             # mutates a in-place

```

Q36. How do you iterate over a dictionary?

Answer: Use `.keys()`, `.values()`, or `.items()`. Iterating the dict directly yields keys.

Example:

```

d = {'a': 1, 'b': 2, 'c': 3}
for k in d:          print(k)          # keys
for v in d.values(): print(v)          # values
for k, v in d.items(): print(k, '->', v) # pairs

```

Q37. How do you sort a dictionary by its values?

Answer: Use `sorted()` with a key function on `dict.items()`. This returns a list of (key, value) tuples; reconstruct with `dict()`.

Example:

```

scores = {'Alice': 88, 'Bob': 95, 'Carol': 72}
sorted_items = sorted(scores.items(), key=lambda kv: kv[1])
# [('Carol', 72), ('Alice', 88), ('Bob', 95)]
sorted_dict = dict(sorted_items)

```

Q38. What is a defaultdict and when would you use it?

Answer: `collections.defaultdict` automatically creates a missing key with a factory value, eliminating `KeyError` checks.

Example:

```

from collections import defaultdict
graph = defaultdict(list)
graph['A'].append('B') # no KeyError even though 'A' is new
graph['A'].append('C')
print(dict(graph)) # {'A': ['B', 'C']}

```

Q39. What is an OrderedDict and is it still needed in Python 3.7+?

Answer: `OrderedDict` maintains insertion order (always). Regular dicts also maintain order since 3.7, but `OrderedDict` has extras: `move_to_end()` and equality is order-sensitive.

Example:

```

from collections import OrderedDict
od = OrderedDict({'a': 1, 'b': 2})
od.move_to_end('a') # move 'a' to end
print(list(od))     # ['b', 'a']

# Order-sensitive equality
print(od == OrderedDict({'a':1,'b':2})) # False

```

Q40. How do you use dictionary comprehensions?

Answer: Similar to list comprehensions but with key: value pairs inside curly braces.

Example:

```
squares = {x: x**2 for x in range(6)}  
# {0:0, 1:1, 2:4, 3:9, 4:16, 5:25}  
  
evens = {k: v for k, v in squares.items() if v % 2 == 0}  
# {0:0, 2:4, 4:16}
```

Q41. How do you safely delete a key from a dictionary?

Answer: Use `del d[key]` (raises `KeyError` if missing) or `d.pop(key, default)` to avoid the error.

Example:

```
d = {'a': 1, 'b': 2}  
del d['a']                # raises KeyError if 'a' absent  
val = d.pop('b', None)    # safe; returns 2, d now {}  
missing = d.pop('x', -1)  # returns -1, no error
```

Q42. What is the difference between `dict.keys()`, `dict.values()`, and `dict.items()`?

Answer: All three return view objects that reflect live changes to the dict. `keys()` → key view, `values()` → value view, `items()` → (key,value) pair view.

Example:

```
d = {'a': 1, 'b': 2}  
kv = d.keys()  
d['c'] = 3  
print(kv)          # dict_keys(['a', 'b', 'c']) live!
```

Q43. How do you count word frequency using a dictionary?

Answer: Use `dict.get()` or `collections.Counter` for a concise approach.

Example:

```
words = ['the', 'fox', 'the', 'dog', 'fox', 'fox']  
freq = {}  
for w in words:  
    freq[w] = freq.get(w, 0) + 1  
print(freq)  # {'the':2, 'fox':3, 'dog':1}  
  
from collections import Counter  
print(Counter(words))  # same result
```

Q44. How do you invert a dictionary (swap keys and values)?

Answer: Use a dict comprehension. If values are not unique, the last key wins (or collect into lists).

Example:

```
original = {'a': 1, 'b': 2, 'c': 3}  
inverted = {v: k for k, v in original.items()}
```

```
print(inverted) # {1:'a', 2:'b', 3:'c'}
```

Q45. What types can be used as dictionary keys?

Answer: Any hashable (immutable) type: int, float, str, tuple (of hashables), frozenset. Lists, dicts, and sets are unhashable and cannot be keys.

Example:

```
d = {}
d[42] = 'int key'
d['hello'] = 'str key'
d[(1, 2)] = 'tuple key'
d[frozenset({1,2})] = 'frozenset key'
# d[[1,2]] = 'x' # TypeError: unhashable type 'list'
```

Q46. How does Python resolve hash collisions in a dictionary?

Answer: Python dicts use open addressing with pseudo-random probing. On collision the probe sequence finds the next available slot.

Example:

```
# Illustrative - CPython hash for small integers is the int itself
print(hash(42)) # 42
print(hash('a')) # platform-specific (randomised in 3.3+)
# Collisions handled internally; user rarely sees them
```

Q47. How do you filter a dictionary based on keys or values?

Answer: Use a dict comprehension with a conditional expression.

Example:

```
d = {'a': 1, 'b': 5, 'c': 3, 'd': 8}
big_vals = {k: v for k, v in d.items() if v > 3}
# {'b': 5, 'd': 8}

keep_keys = {'a', 'c'}
subset = {k: d[k] for k in keep_keys if k in d}
# {'a': 1, 'c': 3}
```

Q48. How do you create a nested dictionary and access its values?

Answer: Nest dicts inside dicts. Use chained `[]` access; use `.get()` chains or dict-path utilities to avoid `KeyErrors`.

Example:

```
company = {
    'engineering': {'Alice': 95, 'Bob': 88},
    'marketing': {'Carol': 91}
}
print(company['engineering']['Alice']) # 95
# Safe access:
score = company.get('hr', {}).get('Dave', 0) # 0
```

Q49. What is ChainMap and when is it useful?

Answer: collections.ChainMap links multiple mappings into a single logical view. Lookups search maps in order. Useful for layered config or scope chains.

Example:

```
from collections import ChainMap
defaults = {'color': 'red', 'size': 10}
overrides = {'color': 'blue'}
cfg = ChainMap(overrides, defaults)
print(cfg['color']) # 'blue' (from overrides)
print(cfg['size']) # 10 (from defaults)
```

Q50. How do you find the key with the maximum value in a dictionary?

Answer: Use max() with key=dict.get.

Example:

```
scores = {'Alice': 88, 'Bob': 95, 'Carol': 72}
winner = max(scores, key=scores.get)
print(winner) # 'Bob'
print(scores[winner]) # 95
```

Q51. How does Python's dict differ from hash maps in other languages?

Answer: CPython's dict is highly optimised: since 3.6 it uses a compact array storing (hash, key, value) triples, keeping insertion order and achieving ~60% memory reduction over the old implementation.

Example:

```
# Python dicts are ordered (guaranteed 3.7+)
d = {}
for i in range(5):
    d[i] = i * 2
print(list(d.keys())) # [0, 1, 2, 3, 4] - insertion order
```

Q52. How do you copy a dictionary?

Answer: dict.copy() / dict(d) create shallow copies. copy.deepcopy() for fully independent copies.

Example:

```
import copy
original = {'a': [1, 2], 'b': 3}
shallow = original.copy()
deep = copy.deepcopy(original)
original['a'].append(99)
print(shallow['a']) # [1, 2, 99] - shared list
print(deep['a']) # [1, 2] - independent
```

Q53. What is the time complexity of dictionary operations?

Answer: Average case: lookup, insert, delete all O(1). Worst case O(n) on pathological hash collisions (rare with hash randomisation).

Example:

```
d = {i: i for i in range(1_000_000)}
# O(1) average:
```

```
val = d[500_000]
d[1_000_001] = 'new'
del d[0]
```

Q54. How can you use a dictionary to implement a switch/case?

Answer: Map keys to callables; call the matched function. Cleaner than long if-elif chains.

Example:

```
def add(a,b): return a + b
def sub(a,b): return a - b
ops = {'+': add, '-': sub}

op, a, b = '+', 10, 3
result = ops.get(op, lambda a,b: None)(a, b)
print(result)    # 13
```

Q55. How do you group items in a list of dicts by a common field?

Answer: Use `itertools.groupby()` after sorting, or build a `defaultdict` of lists.

Example:

```
from collections import defaultdict
records = [
    {'dept': 'Eng', 'name': 'Alice'},
    {'dept': 'Mkt', 'name': 'Bob'},
    {'dept': 'Eng', 'name': 'Carol'},
]
by_dept = defaultdict(list)
for r in records:
    by_dept[r['dept']].append(r['name'])
print(dict(by_dept))
# {'Eng': ['Alice', 'Carol'], 'Mkt': ['Bob']}
```

Q56. What is the difference between `dict.update()` and the `|` operator?

Answer: `dict.update()` mutates in-place and accepts any iterable of (key,value) pairs. `|` (Python 3.9+) returns a new dict.

Example:

```
a = {'x': 1}
b = {'y': 2}
a.update(b)    # a is now {'x':1, 'y':2}, b unchanged

c = {'x': 1}
d = c | b      # new dict {'x':1, 'y':2}, c unchanged
```

Section 3: Sets (Q56–Q75)

Q57. What is a Python set and what are its main properties?

Answer: A set is an unordered collection of unique, hashable elements. It supports $O(1)$ average membership tests and set-algebraic operations.

Example:

```
s = {1, 2, 3, 2, 1}
print(s)           # {1, 2, 3} - duplicates removed
print(type(s))     # <class 'set'>
empty = set()      # NOT {} which creates an empty dict
```

Q58. What are the main differences between a set and a frozenset?

Answer: frozenset is the immutable version of set: it cannot be modified after creation but is hashable, so it can be a dict key or an element of another set.

Example:

```
fs = frozenset([1, 2, 3])
# fs.add(4)           # AttributeError
d = {fs: 'immutable set key'} # valid dict key
print(d[fs])          # 'immutable set key'
```

Q59. How do you perform union, intersection, and difference on sets?

Answer: Use operators (|, &, -) or methods (union, intersection, difference).

Example:

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(a | b)      # {1,2,3,4,5,6}   union
print(a & b)      # {3,4}           intersection
print(a - b)      # {1,2}           difference
print(a ^ b)      # {1,2,5,6}       symmetric difference
```

Q60. How do you add and remove elements from a set?

Answer: add() inserts one element. remove() raises KeyError if missing; discard() does not. pop() removes and returns an arbitrary element.

Example:

```
s = {1, 2, 3}
s.add(4)          # {1,2,3,4}
s.discard(10)     # no error even if absent
s.remove(2)       # {1,3,4}  KeyError if missing
val = s.pop()     # removes arbitrary element
```

Q61. How do you check subset and superset relationships?

Answer: Use issubset() / <= and issuperset() / >= operators.

Example:

```
a = {1, 2}
b = {1, 2, 3}
print(a.issubset(b)) # True   (a <= b)
print(b.issuperset(a)) # True  (b >= a)
print(a < b)          # True   (proper subset)
print(a <= a)         # True   (subset of itself)
```

Q62. Why can't you add a list to a set?

Answer: Sets require hashable elements. Lists are mutable and therefore unhashable. Use a tuple instead.

Example:

```
s = set()
# s.add([1, 2])      # TypeError: unhashable type 'list'
s.add((1, 2))        # OK - tuples are hashable
print(s)              # {(1, 2)}
```

Q63. How do you find common elements between two lists using sets?

Answer: Convert both to sets and take the intersection, then convert back to list if needed.

Example:

```
a = [1, 2, 3, 4, 5]
b = [4, 5, 6, 7, 8]
common = list(set(a) & set(b))
print(common) # [4, 5] (order not guaranteed)
```

Q64. How do you remove duplicates from a list using a set?

Answer: Wrap in set() to deduplicate, then list() to convert back. Note: order is not preserved. Use dict.fromkeys() if order matters.

Example:

```
lst = [3, 1, 4, 1, 5, 9, 2, 6, 5]
no_dups = list(set(lst))
# order-preserving alternative:
no_dups_ordered = list(dict.fromkeys(lst))
```

Q65. What is the time complexity of set operations?

Answer: Average O(1) for add, remove, and 'in'. Union/intersection/difference are O(min(len(s),len(t))) to O(len(s)+len(t)).

Example:

```
big_set = set(range(1_000_000))
# O(1):
999_999 in big_set
big_set.add(1_000_000)
big_set.discard(0)
```

Q66. How do you use set comprehensions?

Answer: Like list comprehensions but with curly braces, producing a set (duplicates automatically removed).

Example:

```
squares = {x**2 for x in range(-5, 6)}
# {0, 1, 4, 9, 16, 25} - each unique value once

vowels = {c for c in 'hello world' if c in 'aeiou'}
# {'e', 'o'}
```

Q67. How do you check if two sets are disjoint?

Answer: Use `set.isdisjoint()` which returns True if the sets share no elements.

Example:

```
a = {1, 2, 3}
b = {4, 5, 6}
c = {3, 7, 8}
print(a.isdisjoint(b)) # True
print(a.isdisjoint(c)) # False (both contain 3)
```

Q68. What is the symmetric difference of two sets?

Answer: Elements in either set but not both. Use `^` operator or `symmetric_difference()`.

Example:

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(a ^ b) # {1, 2, 5, 6}
print(a.symmetric_difference(b)) # {1, 2, 5, 6}
a.symmetric_difference_update(b) # in-place: a = {1, 2, 5, 6}
```

Q69. How do you update a set in-place with another set?

Answer: `update()` / `|=` adds elements; `intersection_update()` / `&=` keeps only common; `difference_update()` / `-=` removes others.

Example:

```
s = {1, 2, 3}
s.update({3, 4, 5}) # s = {1, 2, 3, 4, 5}
s.intersection_update({2, 3, 4}) # s = {2, 3, 4}
s.difference_update({4}) # s = {2, 3}
```

Q70. Can a set contain another set?

Answer: No, because sets are mutable and therefore unhashable. Use `frozenset` as a set element.

Example:

```
inner = frozenset({1, 2})
outer = {inner, frozenset({3, 4})}
print(outer) # {frozenset({1, 2}), frozenset({3, 4})}
```

Q71. How is a Python set implemented internally?

Answer: A set is essentially a dict with keys only (no values). It uses a hash table, so elements must be hashable and lookups are O(1) average.

Example:

```
# Both are hash tables:
s = {1, 2, 3}
d = {1: None, 2: None, 3: None}
# CPython set uses a compact hash table similar to dict
```

Q72. How do you count unique elements using sets?

Answer: Convert to a set and use len().

Example:

```
data = [1, 2, 2, 3, 3, 3, 4]
unique_count = len(set(data))
print(unique_count) # 4
```

Q73. When would you choose a set over a list or dict?

Answer: Choose a set when: (1) you need fast membership testing, (2) elements must be unique, (3) set-algebra operations are needed. Lists preserve order; dicts associate keys to values.

Example:

```
# Use set for fast membership:
valid_users = {'alice', 'bob', 'carol'}
if 'alice' in valid_users: # O(1)
    print('Access granted')
```

Q74. How do you find elements in list A that are not in list B (set difference)?

Answer: Convert both to sets and use the - operator or difference().

Example:

```
A = [1, 2, 3, 4, 5]
B = [3, 4, 5, 6, 7]
only_in_A = list(set(A) - set(B))
print(only_in_A) # [1, 2]
```

Q75. How do you convert between set, list, and tuple?

Answer: Use the built-in constructors: set(), list(), tuple(). They all accept any iterable.

Example:

```
lst = [1, 2, 2, 3]
s = set(lst) # {1, 2, 3}
t = tuple(s) # (1, 2, 3) order not guaranteed
back = list(s) # [1, 2, 3]
```

Q76. How do you use sets for Venn-diagram-style data analysis?

Answer: Use the four set operations (union, intersection, difference, symmetric difference) to model the Venn diagram regions.

Example:

```
python_devs = {'Alice', 'Bob', 'Carol'}
java_devs = {'Bob', 'Dave', 'Carol'}
both = python_devs & java_devs # {'Bob', 'Carol'}
only_py = python_devs - java_devs # {'Alice'}
either = python_devs | java_devs # all 4 names
```

Section 4: Tuples (Q76–Q100)

Q77. What is a Python tuple and when should you use it instead of a list?

Answer: A tuple is an immutable ordered sequence. Use tuples for fixed collections (coordinates, RGB, DB records), function return values, and dict keys. They use less memory than lists.

Example:

```
point = (3, 5)          # immutable (x, y)
rgb    = (255, 128, 0)  # colour constant
# point[0] = 9          # TypeError
import sys
print(sys.getsizeof([1,2,3])) # ~88 bytes
print(sys.getsizeof((1,2,3))) # ~72 bytes
```

Q78. How do you create a tuple with a single element?

Answer: A trailing comma is required; without it, parentheses are just grouping.

Example:

```
single = (42,)          # tuple of one element
not_tuple = (42)         # just int 42
print(type(single))      # <class 'tuple'>
print(type(not_tuple))   # <class 'int'>
```

Q79. What is tuple packing and unpacking?

Answer: Packing: multiple values are automatically packed into a tuple. Unpacking: a tuple is expanded into multiple variables.

Example:

```
packed = 1, 2, 3         # tuple (1, 2, 3) - no parens needed
a, b, c = packed          # unpacking
print(a, b, c)            # 1 2 3

# Swap two variables:
x, y = 10, 20
x, y = y, x
print(x, y)               # 20 10
```

Q80. How does extended iterable unpacking work with tuples?

Answer: The starred (*) target collects any leftover elements into a list.

Example:

```
first, *rest = (1, 2, 3, 4, 5)
print(first)   # 1
print(rest)    # [2, 3, 4, 5]

*init, last = (1, 2, 3, 4, 5)
print(init)    # [1, 2, 3, 4]
print(last)    # 5
```

Q81. Why are tuples faster than lists?

Answer: Tuples are allocated as a single fixed-size block; CPython even caches small tuples. Lists need an extra layer of indirection for dynamic resizing.

Example:

```
import timeit
t_list = timeit.timeit('x=[1,2,3,4,5]', number=10_000_000)
t_tuple = timeit.timeit('x=(1,2,3,4,5)', number=10_000_000)
print(f'list: {t_list:.2f}s  tuple: {t_tuple:.2f}s')
# tuple is typically ~2x faster to create
```

Q82. Can a tuple contain mutable objects?

Answer: Yes. The tuple itself cannot be reassigned, but the mutable objects it contains can still be modified.

Example:

```
t = ([1, 2], [3, 4])
t[0].append(99)      # modifying the list inside - OK
print(t)             # ([1, 2, 99], [3, 4])
# t[0] = []          # TypeError - cannot reassign element
```

Q83. How do you use a tuple as a dictionary key?

Answer: Tuples of hashable elements are hashable, so they can be dict keys (unlike lists).

Example:

```
grid = {}
grid[(0, 0)] = 'origin'
grid[(1, 2)] = 'point'
print(grid[(0, 0)]) # 'origin'
```

Q84. What is a named tuple and how do you create one?

Answer: `collections.namedtuple` creates a tuple subclass with named fields, combining tuple immutability with attribute-style access.

Example:

```
from collections import namedtuple
Point = namedtuple('Point', ['x', 'y'])
p = Point(3, 5)
print(p.x, p.y)    # 3 5
print(p[0])        # 3 (index access still works)
x, y = p           # unpacking still works
```

Q85. How is `typing.NamedTuple` different from `collections.namedtuple`?

Answer: `typing.NamedTuple` uses class syntax with type annotations and supports default values. `collections.namedtuple` is functional and terse.

Example:

```
from typing import NamedTuple
class Employee(NamedTuple):
```

```
name: str
dept: str
salary: float = 50_000.0

e = Employee('Alice', 'Eng')
print(e.salary)    # 50000.0
```

Q86. How do you concatenate and repeat tuples?

Answer: Use + to concatenate and * to repeat. Both return new tuples.

Example:

```
a = (1, 2, 3)
b = (4, 5)
print(a + b)    # (1, 2, 3, 4, 5)
print(a * 3)    # (1, 2, 3, 1, 2, 3, 1, 2, 3)
```

Q87. How do you convert a tuple to a list and vice versa?

Answer: Use list() and tuple() constructors.

Example:

```
t = (1, 2, 3)
lst = list(t)    # [1, 2, 3]
back = tuple(lst) # (1, 2, 3)
```

Q88. What methods does a tuple support?

Answer: Only count() and index() – far fewer than list because tuples are immutable.

Example:

```
t = (1, 2, 3, 2, 4, 2)
print(t.count(2))    # 3
print(t.index(3))    # 2 (first occurrence)
```

Q89. How do tuples behave in function arguments and return values?

Answer: Functions naturally return multiple values as tuples. *args collects positional arguments as a tuple.

Example:

```
def minmax(lst):
    return min(lst), max(lst) # returns a tuple

lo, hi = minmax([4, 1, 7, 2])
print(lo, hi)    # 1 7

def show(*args):
    print(type(args), args)    # <class 'tuple'>
```

Q90. How do you sort a list of tuples?

Answer: `sorted()` / `list.sort()` with a `key=` function. Tuples also compare lexicographically by default.

Example:

```
records = [('Bob', 25), ('Alice', 30), ('Carol', 22)]
by_name = sorted(records)           # alphabetical by first elem
by_age = sorted(records, key=lambda r: r[1])
print(by_age) # [('Carol', 22), ('Bob', 25), ('Alice', 30)]
```

Q91. Can you iterate over a tuple?

Answer: Yes, tuples support all iteration protocols: for loops, `enumerate()`, `zip()`, and comprehensions.

Example:

```
colours = ('red', 'green', 'blue')
for i, c in enumerate(colours):
    print(i, c)
# 0 red 1 green 2 blue

upper = tuple(c.upper() for c in colours)
# ('RED', 'GREEN', 'BLUE')
```

Q92. How do you compare two tuples?

Answer: Tuples are compared lexicographically: element-by-element from left to right.

Example:

```
print((1, 2, 3) < (1, 2, 4)) # True (3 < 4 at index 2)
print((1, 3) > (1, 2, 99)) # True (3 > 2 at index 1)
print((1, 2) == (1, 2)) # True
```

Q93. How do you check membership in a tuple?

Answer: Use the `'in'` operator; it performs a linear $O(n)$ scan.

Example:

```
t = (1, 2, 3, 4, 5)
print(3 in t) # True
print(9 in t) # False
```

Q94. What is tuple hashing and why does it matter?

Answer: A tuple is hashable when all its elements are hashable, making it usable as a dict key or set element. The hash is derived from all elements.

Example:

```
t = (1, 'hello', 3.14)
print(hash(t)) # some integer

# Useful as compound dict key:
cache = {}
cache[(2, 'en')] = 'translated'
```

Q95. How do you use tuples in data classes or records patterns?

Answer: Use namedtuple or dataclass for structured records. Tuples shine as lightweight, positional record types.

Example:

```
from collections import namedtuple
Stock = namedtuple('Stock', 'symbol price volume')
aapl = Stock('AAPL', 189.5, 1_234_567)
print(aapl.symbol, aapl.price) # AAPL 189.5
# Can be used as dict key (immutable):
prices = {aapl: 'buy'}
```

Q96. When should you NOT use a tuple and prefer a list?

Answer: Use a list when: the collection needs to grow or shrink, elements need to be modified, the order changes, or you need list-specific methods (sort, append, etc.).

Example:

```
# Queue that grows at runtime - use list (or deque)
queue = []
queue.append('task1')
queue.append('task2')
queue.pop(0)

# Fixed record - tuple is better
ORIGIN = (0, 0) # never changes
```